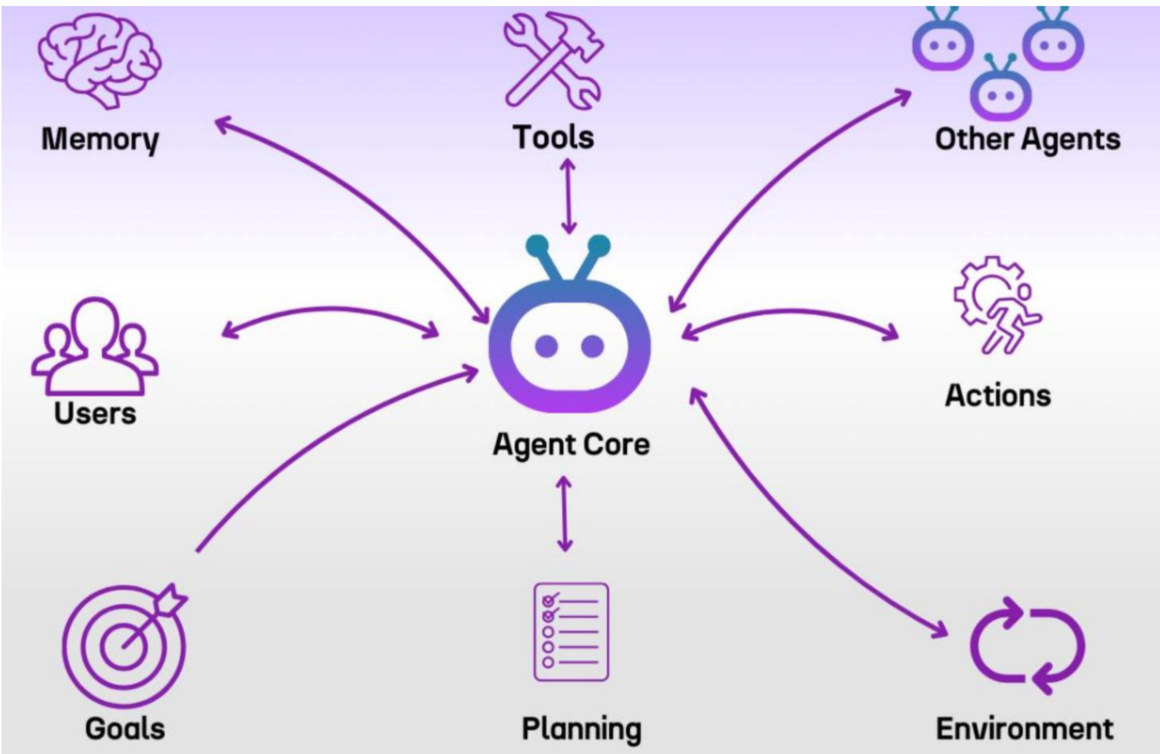
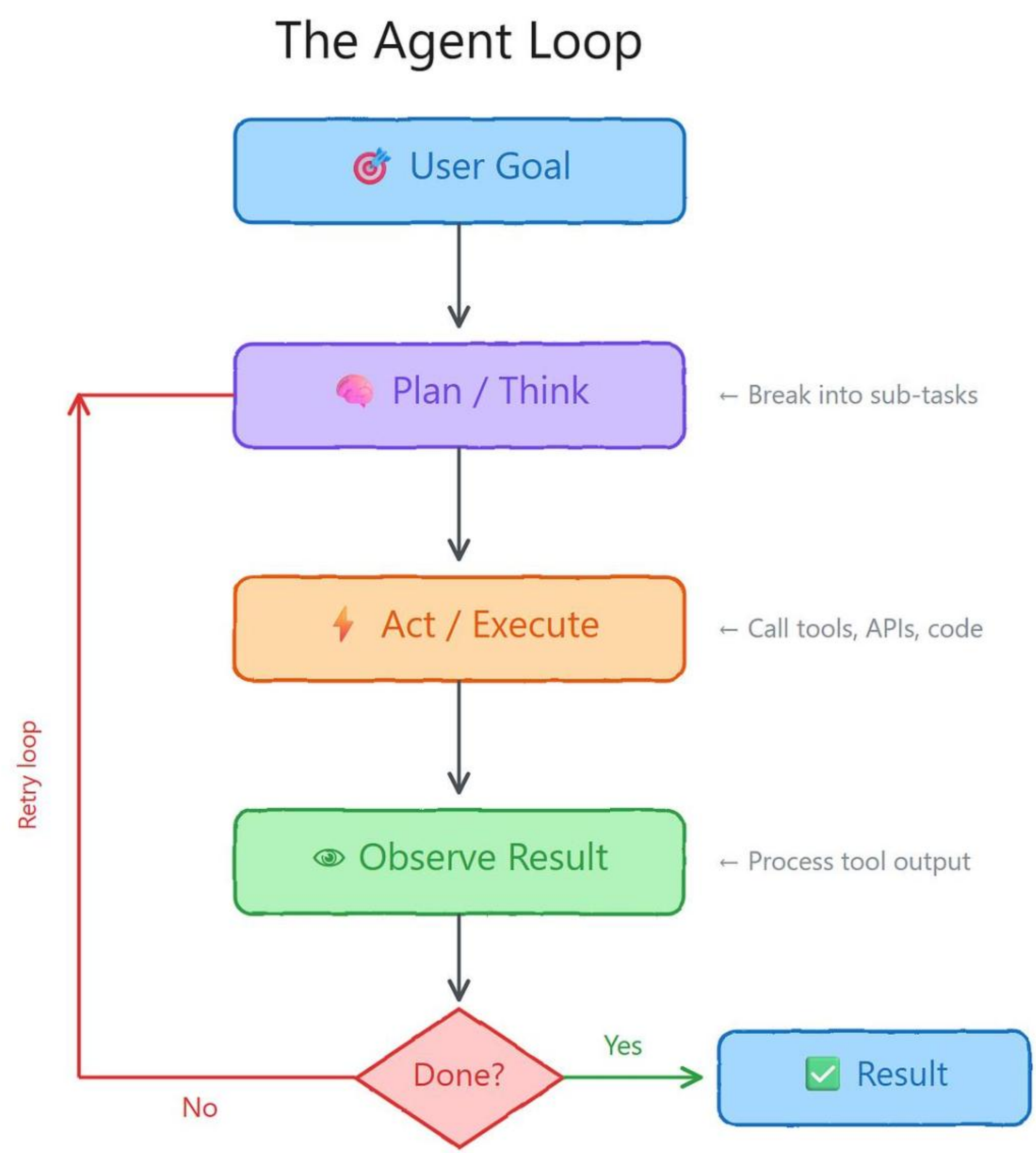
“Building Agentic AI Assistants: A Tutorial for Everyone”

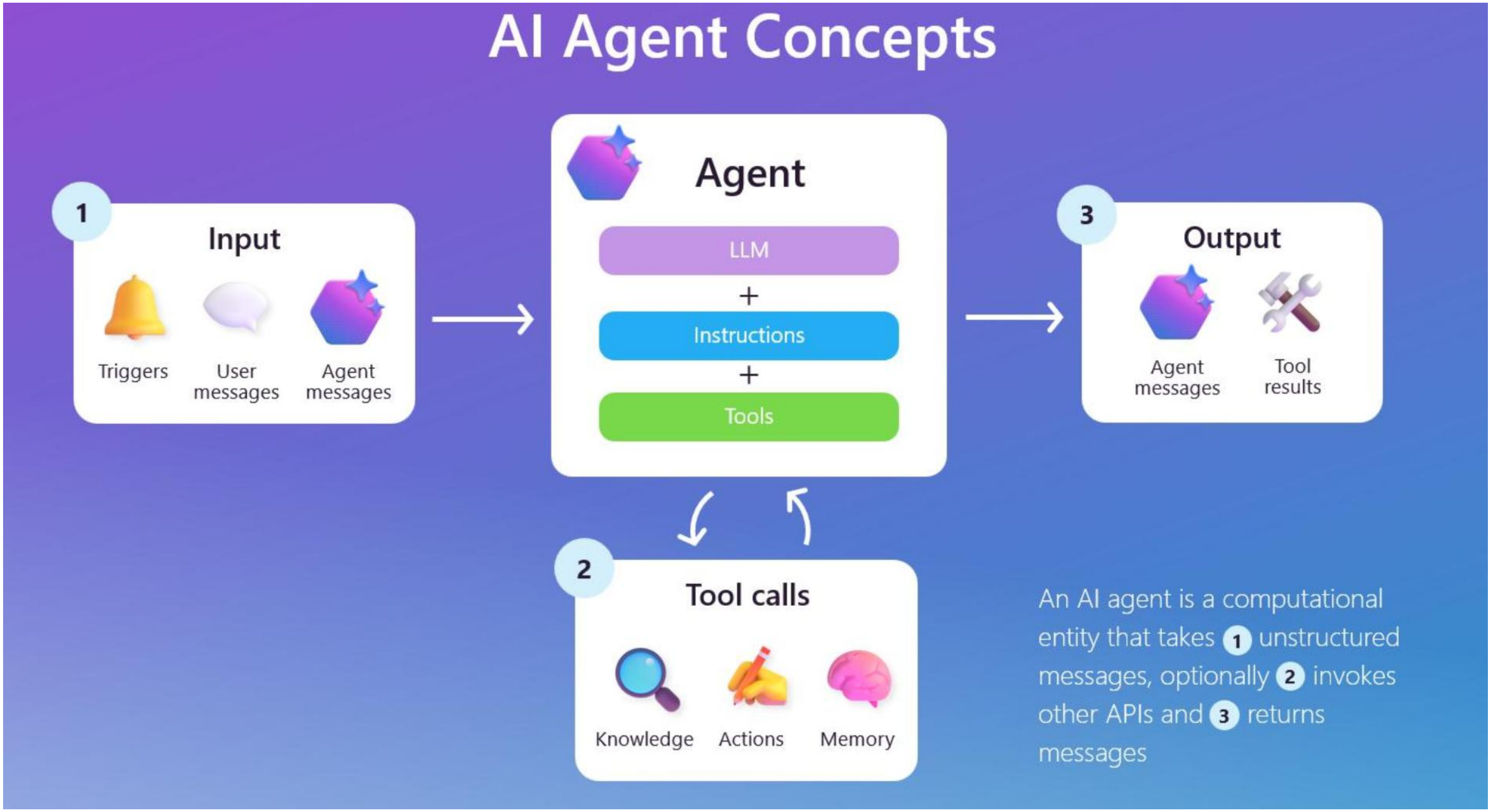
Ernest Bonat, Ph.D.
ernest point bonat at gmail point com

1. What Is Agentic AI?

Agentic AI systems are **goal-driven, autonomous AI systems** that can reason, plan, and use tools to complete tasks.




The Agent Loop
🎯 User Goal
🧠 Plan / Think
← Break into sub-tasks
⚡ Act / Execute
← Call tools, APIs, code
👁 Observe Result
← Process tool output
Done?
Yes
✅ Result
No
Retry loop


AI Agent Concepts
1
Input
Triggers
User messages
Agent messages
Agent
LLM
+
Instructions
+
Tools
3
Output
Agent messages
Tool results
2
Tool calls
Knowledge
Actions
Memory
An AI agent is a computational entity that takes 1 unstructured messages, optionally 2 invokes other APIs and 3 returns messages

Key Differences

Traditional AI	Prompt-Response AI	Agentic AI
Static models	One-shot answers	Multi-step reasoning
No memory	Stateless	Memory-enabled
No tools	Limited tools	Dynamic tool use
Deterministic	Reactive	Adaptive & goal-oriented

Agentic AI is not just answering—it is **acting, deciding, and iterating**.

2. Core Components & Architecture

Core Layers

1. **LLM Brain**
   - Reasoning engine (e.g., GPT, LLaMA, Claude)
2. **Memory**
   - Short-term (conversation state)
   - Long-term (vector DB, knowledge base)
3. **Tools (Actions)**
   - APIs, databases, Python functions
   - Bioinformatics tools (BLAST, PubMed, etc.)
4. **Planning**
   - Break tasks into steps
   - Decide which tools to use

5. **Execution**
    - Perform tool calls
    - Collect results
6. **Orchestration**
    - Manage workflow (LangGraph, workflows)

“The future is not just faster pipelines — but systems that understand what they compute.”

3. Frameworks & Tools (2024–2025)

Leading Ecosystem

Category	Tools
Orchestration	LangGraph, LangChain
LLM APIs	OpenAI, Anthropic, Google
Vector DB	FAISS, Chroma
Evaluation	RAGAS
Optimization	Optuna
Explainability	SHAP
Deployment	FastAPI, Streamlit

Emerging Standard

- **MCP (Model Context Protocol)** → structured connection to:
    1. EHR systems
    2. PubMed
    3. SQL databases

Trend: Shift from **pipelines** → **orchestrated intelligent systems**

4. Agent Skills – The New Paradigm

“Instead of building one large agent: Build modular skills”

Examples:

- 🧬 DNA sequence classifier
- 📊 Statistical analysis tool
- 🔎 PubMed search skill
- 🏥 Clinical rules validator

Architecture Insight

Agent = Orchestrator
Skill = Capability
Tool = Implementation

🧠 Agent (Orchestrator)

The “brain” that decides *what to do next*

- Plans tasks
- Chooses which skill to use
- Coordinates execution

⚙️ Skill (Capability)

A reusable **functional module**

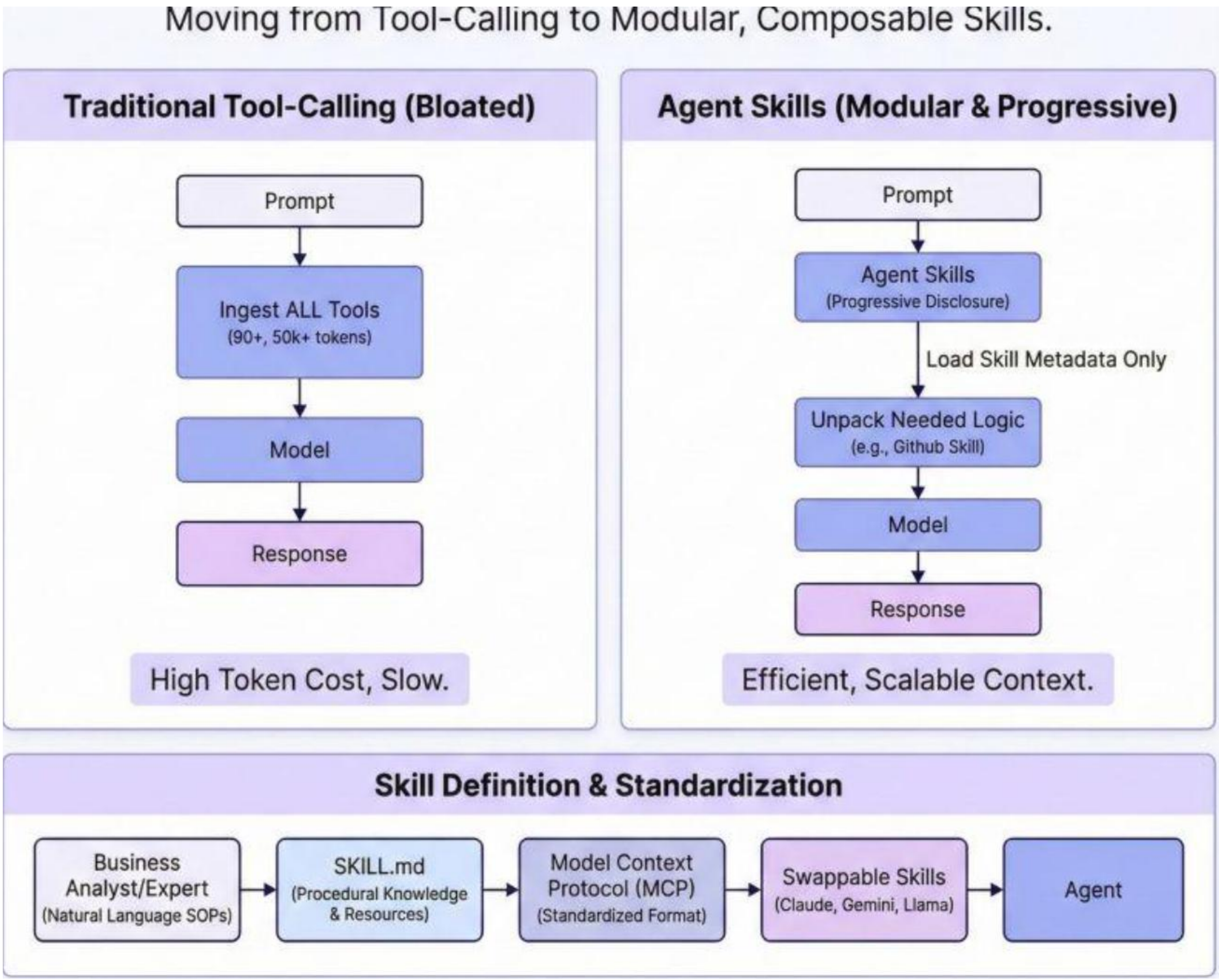
- Encapsulates logic (e.g., DNA analysis, PubMed search)

- Abstracts complexity from the agent

🔧 Tool (Implementation)

The actual **code or API** doing the work

- Python functions
- External APIs
- Databases / ML models


Moving from Tool-Calling to Modular, Composable Skills.
Traditional Tool-Calling (Bloated)
Prompt
Ingest ALL Tools
(90+, 50k+ tokens)
Model
Response
High Token Cost, Slow.
Agent Skills (Modular & Progressive)
Prompt
Agent Skills
(Progressive Disclosure)
Load Skill Metadata Only
Unpack Needed Logic
(e.g., Github Skill)
Model
Response
Efficient, Scalable Context.
Skill Definition & Standardization
Business Analyst/Expert
(Natural Language SOPs)
SKILL.md
(Procedural Knowledge & Resources)
Model Context Protocol (MCP)
(Standardized Format)
Swappable Skills
(Claude, Gemini, Llama)
Agent

5. Agent AI – The Core Problem Everywhere!

Pipeline Process -> DETERMINISTIC (same result every time)
Agent AI -> PROBABILISTIC (result probably correct)

The Determinism Spectrum

```
DETERMINISTIC ◄──────────────────────────────► PROBABILISTIC

  Pipeline            Agent              Agent + Skills
  (Python)            (LLM, temp=0)      (LLM, temp>0)

  Same output         Usually same       Different output
  every single        output, but        every run —
  run, forever        not guaranteed     by design
```

4. The Risk Matrix

Scenario	Pipeline Risk	Agent Risk	Verdict
ACMG variant classification	Low — rules are coded	HIGH — LLM interprets evidence	Pipelines only
Clinical lab report generation	Low — templates	HIGH — LLM writes free text	Pipelines only
Drug-drug interaction check	Low — lookup table	HIGH — LLM may miss edge cases	Pipelines only
NGS QC (FastQC thresholds)	Low — fixed thresholds	Medium — LLM may vary cutoffs	Pipelines preferred
Literature search strategy	Medium — fixed queries	Low — LLM adapts better	Agents acceptable

Scenario	Pipeline Risk	Agent Risk	Verdict
Hypothesis generation for EDA	Medium — hardcoded list	Low — LLM produces better hypotheses	Agents preferred
Drafting clinical summary text	N/A — pipelines can't	Medium — LLM is adequate	Agents with review

ernest.bonat@gmail.com